Research Square

Preprints are preliminary reports that have not undergone peer review.
They should not be considered conclusive, used to inform clinical practice,
or referenced by the media as validated information.

PEBSI: Policy-Efficient Branching Variable Selection via Reinforcement Learning

Shuhan Du
Tsinghua University
Junbo Tong
Tsinghua University
Daming Shi
shidm18@tsinghua.org.cn
Tsinghua University
Yi Liu
Tsinghua University
Wenhui Fan
Tsinghua University

Research Article

Keywords: Mixed Integer Linear Programs, Branch-and-bound, Learning-based branching, Reinforcement Learning

Posted Date: November 26th, 2025

DOI: https://doi.org/10.21203/rs.3.rs-7863374/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License.
Read Full License

Additional Declarations: No competing interests reported.

PEBSI: Policy-Efficient Branching Variable Selection via Reinforcement Learning

Shuhan Du[1], Junbo Tong[1], Daming Shi[1*], Yi Liu[1], Wenhui Fan[1]

[1*]Department of Automation, Tsinghua University, Shuangqing Road, Beijing, 100084, Beijing, China.

*Corresponding author(s). E-mail(s): shidm18@tsinghua.org.cn;
Contributing authors: dsh24@mails.tsinghua.edu.cn;
tjb21@mails.tsinghua.edu.cn; yiliu@tsinghua.edu.cn;
fanwenhui@tsinghua.edu.cn;

Abstract

Mixed Integer Linear Programs (MILPs) are widely used to model real-world optimization problems, with the branch-and-bound (B&B) algorithm serving as a fundamental solution method. The choice of branching strategy is crucial, and recent advances in machine learning have fueled interest in learning-to-branch techniques. Although imitation learning (IL) has shown promising results in replicating handcrafted branching heuristics, its performance is fundamentally limited by the quality of the expert strategy and the high cost of data labeling. Reinforcement learning (RL) presents a promising alternative, but the complexity of the branching process poses significant challenges for its direct application. Consequently, many existing RL-based methods still rely on expert demonstrations for pretraining or data augmentation, or impose additional constraints on the training procedure. This work introduces PEBSI, an efficient RL-based branching policy that eliminates the need for expert demonstrations or auxiliary configurations. The RL agent is trained using retrospective trajectories constructed from the original B&B search tree, with a novel reward function and exploration strategy designed to improve policy efficiency. Experimental evaluations on diverse MILP benchmarks demonstrate that PEBSI outperforms other RL-based methods trained without expert guidance and, in some cases, even surpasses the IL-based approach.

Keywords: Mixed Integer Linear Programs, Branch-and-bound, Learning-based branching, Reinforcement Learning

1 Introduction

A Mixed Integer Linear Program (MILP) is an optimization problem with a linear objective function subject to linear constraints, where some variables are restricted to integer values. MILPs naturally arise in a wide range of decision-making problems, including process scheduling [1], production planning [2], vehicle routing [3], and traffic management [4]. Solving MILPs has long been a challenge, leading to the development of numerous algorithms, among which the branch-and-bound (B&B) algorithm [5] stands out as a widely adopted framework. B&B systematically partitions the solution space into a tree structure, computing relaxation bounds at each step until an optimal integer solution is identified. Enhanced by various techniques to improve efficiency, B&B serves as a cornerstone of modern MILP solvers [6]. The algorithm's divide-and-conquer nature requires making sequential decisions: at each iteration, it selects a node from the search tree to process and branches on a variable to create subproblems. While both node selection and variable selection influence how the search tree evolves, this work focuses exclusively on branching variable selection, a key factor in determining the tree's size [7].

Traditional variable selection rules often rely on intuition and empirical evidence [8], with Strong Branching (SB) [9] being a notable example that typically produces smaller search trees than other expert-crafted rules at the cost of extensive computation. More recently, machine learning (ML) has emerged as a compelling alternative, enabling the development of data-driven branching strategies. This has led to a well-established body of work [10–15] that employs imitation learning (IL) to approximate the decision-making process of SB. However, no universally optimal branching policy exists for all problem types, and SB is not always the best choice. It prioritizes local one-step look-ahead dual-bound improvements, which can sometimes lead to suboptimal decisions by neglecting the global impact on deeper levels of the B&B tree [16]. Additionally, it may struggle to provide reliable demonstrations when the linear programming (LP) relaxation is uninformative or suffers from dual degeneracy [17, 18]. The limitations of the expert strategy can hinder the effectiveness of the learned IL policy. Furthermore, obtaining IL training data is non-trivial due to the high computational cost of executing the expert strategy.

Reinforcement learning (RL) has emerged as a promising approach to address these challenges, offering the potential to derive effective branching strategies without expert supervision. However, training an RL branching policy from scratch remains difficult in practice due to long episode durations, a large state-action space, and partial observability [19]. As a result, most existing studies still rely on expert demonstrations for model pretraining or sample augmentation [20–24]. Etheve et al. [25] were the first to propose an RL-based branching policy learned without expert guidance, but their approach requires using a specific, suboptimal node selection rule during training. Scavuzzo et al. [26] extended their work and proposed a different method to eliminate the reliance on expert experience, but it requires solving training instances in advance. These additional restrictions are often impractical in real-world settings.

This work presents PEBSI, a novel RL-based approach for learning a variable selection strategy within the B&B framework that removes the need for expert demonstrations or additional constraints during training. Building on the insights of

Parsonson et al. [19], the RL agent is trained by learning from retrospectively constructed trajectories of the full B&B search tree. A carefully designed reward function, along with an exploration strategy, is proposed to enhance performance. Experimental results demonstrate PEBSI's superior performance in policy efficiency over other RL-based methods trained from scratch [19, 25, 26]. In some cases, it even exceeds the IL-based method [13]. In summary, the contributions of this work are threefold:

- Propose an efficient RL-based branching policy without requiring expert demonstrations or auxiliary training settings.
- Propose a reward function to capture the varying impact of agent actions and encourage effective decisions during solving.
- Propose an exploration strategy to reduce uninformative trajectories and improve the quality of training samples.

The remainder of this paper is organized as follows. Section 2 introduces the necessary preliminaries. Section 3 provides a brief review of the related literature. Section 4 outlines the proposed approach. Section 5 describes the experimental setup and the results. Finally, Section 6 concludes the paper and discusses potential directions for future research.

2 Background

2.1 The B&B Algorithm

A general MILP can be defined as

$$\min\{\mathbf{c}^T\mathbf{x} : \mathbf{A}\mathbf{x} \leq \mathbf{b}, \mathbf{x} \in \mathbb{Z}^k \times \mathbb{R}^{n-k}\}, \tag{1}$$

where $\mathbf{c} \in \mathbb{R}^n$, $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$, $\mathbf{c}^T\mathbf{x}$ is the objective function, and $k \leq n$ is the number of integer variables. B&B is a classic tree search algorithm that iteratively partitions the feasible set of the original problem (1) into disjoint subproblems and computes bounds for these subproblems to rule out unpromising regions.

The algorithm maintains a list of unexplored leaf nodes, known as *open nodes*, initialized with the root node representing the original problem. At each iteration, B&B selects an open node based on a predefined strategy, a decision process referred to as the *node selection problem*. Once a node is selected, the algorithm relaxes the integer constraints and solves the resulting LP relaxation, a convex optimization problem [27]. The optimal solution, denoted as $\mathbf{x}^*$, provides a lower bound (also known as *dual bound*) for the original problem. If $\mathbf{x}^*$ also satisfies the integer constraints, it constitutes a feasible solution and provides an upper bound. Otherwise, the fractional variables in $\mathbf{x}^*$, also called *candidates*, are used to further partition the search space. The *variable selection problem* consists of choosing a candidate $x_j \in \mathbf{x}^*$ ($j \in \{1, \ldots, k\}$) to branch on, creating child nodes according to the split

$$x_j \leq \lfloor {x^*}_j \rfloor \vee x_j \geq \lceil {x^*}_j \rceil. \tag{2}$$

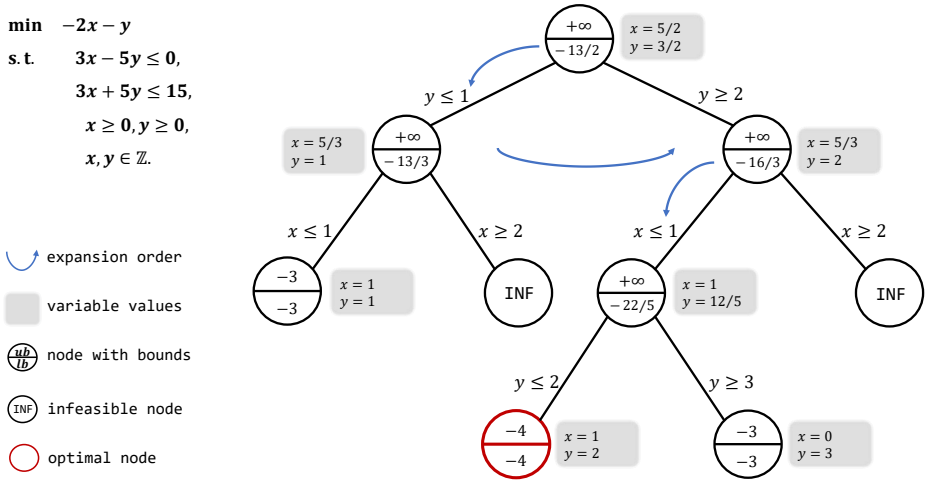


Fig. 1 Using B&B to solve an MILP instance, where ub denotes the upper bound and lb denotes the lower bound

If the lower bound of a node exceeds the objective value of the current best feasible solution, the bounding mechanism prunes the node, as no solution within its subtree can be optimal. An example illustrating the application of B&B to solve an MILP instance is provided in Figure 1. This work focuses on the critical task of variable selection, as the quality of the branching strategy dramatically influences the search process and the overall efficiency of B&B.

2.2 Traditional branching heuristics

While there is no universal metric to measure the efficiency of a branching policy, a common evaluation criterion is the final size of the B&B tree [28], as it reflects the policy's decision-making quality at each step. The literature offers a variety of branching rules, with SB empirically known to produce significantly smaller search trees than other heuristics. However, SB achieves this by calculating the expected one-step dual bound improvement for each candidate before branching, which can be myopic, as it neglects long-term effects on deeper levels of the tree, potentially leading to suboptimal outcomes [16]. Moreover, when the LP relaxation is uninformative or exhibits dual degeneracy (i.e., multiple simplex bases yield the same dual solution) [17, 18], SB may fail to provide a meaningful selection criterion. Its high computational cost further limits practical deployment at every node.

In practice, solvers employ more efficient branching strategies. One such method is pseudo-cost branching (PC) [29], which tracks the impact of past branchings on LP relaxation values for each variable. With sufficient historical data, PC can approximate SB's gains using only a few arithmetic operations. Despite its efficiency, PC's performance is highly sensitive to proper initialization. Recent advances, such as hybrid

branching [30], combine the strengths of SB and PC by applying SB in the early stages and later switching to faster heuristics like PC. For a comprehensive review of traditional branching rules, readers are referred to [16].

2.3 Markov decision process (MDP) formulation

RL is typically formalized through the framework of MDP, which can be defined by $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$. Here, $\mathcal{S}$ represents the state space, $\mathcal{A}$ the action space, $\mathcal{T}$ the transition function, $\mathcal{R}$ the reward function, and $\gamma \in [0, 1]$ the discount factor. At each time step t, selecting an action $a_t \in \mathcal{A}$ transitions the system from $s_t \in \mathcal{S}$ to $s_{t+1} \in \mathcal{S}$ according to $\mathcal{T}$, yielding a reward r_t based on $\mathcal{R}$. The discount factor γ balances immediate rewards and long-term gains. Given a training dataset $\mathcal{D}$ consisting of samples in the form (s_t, a_t, s_{t+1}, r_t), the RL agent learns a policy π using specific algorithms.

In the context of branching variable selection, the MDP state encodes the current status of the search tree, often represented as a bipartite graph [13]. The action space consists of candidate variables, and selecting an action corresponds to choosing a variable to branch on. State transitions are determined by the solver's node selection rule, which is separate from variable selection, making the transitions only partially observable to the brancher. Details on the reward function design of this work are provided in Section 4.

3 Related works

Developing an effective branching strategy is essential for enhancing the performance of modern MILP solvers. By leveraging the vast amount of data generated during the optimization process, learn-to-branch methods have gained significant attention, with using IL to mimic the behavior of SB emerging as the most extensively studied approach. Early studies in this area employed fixed-dimensional representations [10–12]. Gasse et al. [13] later advanced this approach by introducing a bipartite graph representation and a policy model based on Graph Convolutional Networks (GCNs). Building on their work, subsequent research further expanded the field [14, 15, 31]. However, as previously discussed, the effectiveness of IL is inherently constrained by the expert strategy and requires costly data labeling. RL approaches are recognized for the ability to learn an optimal policy directly with respect to any desired metric, driving growing interest in their application to this field.

Sun et al. [20] proposed a primal-dual policy network (PD policy) and trained the RL agent using a novelty search evolution strategy. They initialized the policy in the same way as [13] with less training data, otherwise the model can hardly solve an instance in a reasonable time. Qu et al. [21] also employed a supervised learning approach as a warm start for their model. The pretrained model can be used to produce informative self-generated samples, which were mixed with expert demonstrations for policy learning. Zhang et al. [22], on the other hand, proposed an algorithm called HRL-Aug, which mixes the samples adaptively and dynamically using an online RL agent. Huang et al. [24] highlighted the importance of optimizing the utility of the entire MILP solving process rather than focusing narrowly on individual steps. They proposed Branch Ranking (BR), formulated the branching task as an offline RL

problem, constructing a dataset that combined samples from SB demonstrations with those from rollout trajectories (referred to as short-term and long-term information, respectively). Similarly, Zhang et al. [23] emphasized the utility of the search tree by incorporating Monte-Carlo tree search (MCTS) [32] in policy optimization. They proposed PPO-MCTS, which also used IL to pretrain the RL agent. Although these studies have explored RL to improve performance, they still rely on expert demonstrations for pretraining or data augmentation, missing RL's long-term promise of learning novel policies independently from human-designed guidance and potentially surpassing expert experience.

Etheve et al. [25] were the first to learn a branching policy without expert guidance. They proposed an algorithm called FMSTS, demonstrating that under a depth-first search (DFS) node selection rule [33], minimizing the size of each subtree is equivalent to minimizing the size of the whole tree. Building on this insight, they developed a value-based RL procedure to learn the policy. However, pure DFS can be significantly less effective than the default node selection strategy used in most MILP solvers [29]. Scavuzzo et al. [26] revisited this approach and proposed a framework termed tree Markov Decision Processes (tMDP). They argued that enforcing DFS for node selection and providing the optimal objective limit during training can both trigger the tree Markov property, enabling the use of policy gradient methods for optimization. However, similar to the DFS requirement, the need for presolving training instances remains impractical in real-world applications. Parsonson et al. [19] proposed retro branching (Retro), which learns from retrospective trajectories of the search tree using Deep Q-Network (DQN) [34], eliminating additional restrictions on the training procedure. However, their approach assigns equal credit to all steps that do not lead to a fathomed node, overlooking the varying impact of different actions. Furthermore, their reliance on simple stochastic action sampling for exploration may struggle to effectively navigate the vast state-action space and complex sequential decision-making inherent in branching.

The related RL-based approaches are summarized in Table 1. The proposed PEBSI learns without relying on expert demonstrations or auxiliary training settings. It captures the intrinsic impact of branching decisions on the solving process through a tailored reward function. It also employs a novel exploration strategy that filters out uninformative trajectories and enhances the quality of training samples.

4 Methodology

4.1 Retrospective trajectories

Applying RL directly from scratch can be computationally intractable considering the exponential size of the entire search tree. To address this challenge, the approach of learning from retrospectively constructed trajectories [19] is adopted. This paradigm offers a general framework compatible with various RL algorithms. An illustration of the pipeline is presented in Figure 2.

The RL brancher is first applied to the training instance to generate an initial episode, forming the original search tree. After solving the instance, rather than

Table 1 Comparison of different RL branching methods. **Free from Expert**: Does not require expert demonstrations for pretraining or data augmentation. **Self-contained Framework**: Does not rely on specific configurations of other solver components (e.g., node selection rules or optimal objective limit settings) during training. **Decision-aware Reward**: The reward signal captures the intrinsic impact of branching decisions on the solving process. **Guided Sampling**: Refines the quality of training samples by leveraging contextual guidance

Methods	Free from Expert	Self-contained Framework	Decision-aware Reward	Guided Sampling
PD Policy[20]		✓		✓
Expert-augmented RL[21]		✓	✓	✓
HRL-Aug[22]		✓	✓	✓
BR[24]		✓	✓	
PPO-MCTS[23]		✓	✓	✓
FMSTS [25]	✓			
tMDP [26]	✓			
Retro [19]	✓	✓		
PEBSI (Ours)	✓	✓	✓	✓

directly using the observed MDP transitions for model optimization, multiple retrospective trajectories are constructed from the original tree. This construction process iteratively links the highest-level available node (i.e., not yet assigned to any trajectory) to an unselected fathomed leaf within its subtree, ensuring that each node is included in exactly one trajectory. The transitions within each retrospective trajectory are then utilized for learning.

This approach eliminates the need for using DFS node selection strategy or presolving the instances during training. By restricting sequential nodes to the same subtree, it also enhances observability, as the characteristic changes in the MDP states become more closely linked to the branching actions. Furthermore, the episode length is significantly reduced compared to the size of the full tree, enabling the agent to learn effectively without expert guidance.

Following [13], the states are encoded using the classic bipartite graph representation, and the network architecture consists of a series of GCN modules, followed by a final Multilayer Perceptron (MLP) module. The processed embeddings are then passed through a softmax activation function to produce a probability distribution over the candidates.

4.2 Reward function

As shown in [19], reconstructing the original search tree enables the use of a simple distance-to-goal reward function, which would otherwise be ineffective over the full episode. Specifically, as defined in (3), a penalty of -1 is assigned at each step t, except when the action leads to the end of the retrospective trajectory, where the reward is 0.

$$r_t^d = \begin{cases} 0 & \text{if } s_{t+1} \text{ reaches a fathomed node,} \\ -1 & \text{otherwise.} \end{cases} \tag{3}$$

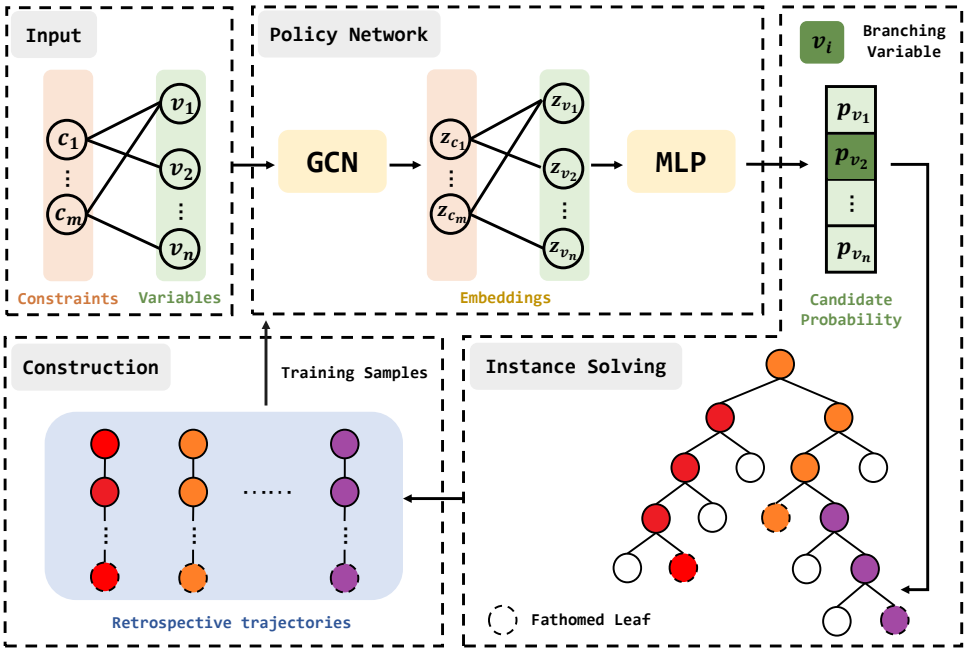
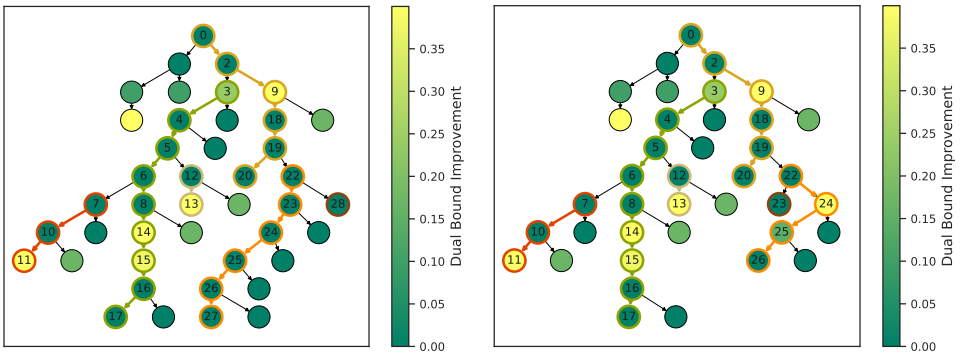


Fig. 2 Learn to branch with retrospective trajectories instead of the original full B&B episodes

While this reward function straightforwardly encourages the agent to reach terminal states efficiently, it fails to account for the varying impact of different actions on the solving process. Figure 3(a) illustrates a B&B search tree for a solved MILP instance using a brancher trained with (3). The figure depicts the one-step dual bound improvement for each node (relative to its parent node, normalized by the overall dual bound improvement during solving). It is evident that branching actions differ significantly in their impact on improving the dual bound, which plays a crucial role in narrowing the search space, simplifying the search process, and ultimately terminating the optimization.

To better leverage the rich data provided by the solver and enhance learning efficiency, a dual bound improvement bonus is introduced in PEBSI, which encourages the agent to take meaningful actions that contribute to improving the dual bound. To mitigate short-sighted behavior, the bonus balances immediate improvements from the previous state with cumulative improvements over the trajectory, controlled by a weighting parameter. Without loss of generality, considering minimization problems, the bonus at step t is formally defined as

$$r_t^b = \frac{(1-\lambda)(DB(s_{t+1}) - DB(s_t)) + \lambda(DB(s_e) - DB(s_{t+1}))}{DB_b - DB_w}, \tag{4}$$



(a) (b)

Fig. 3 Illustrations of the reward function and the exploration strategy. (a) The B&B search tree for solving a specific MILP using a branching agent trained with (3), where node colors represent the normalized dual bound improvements relative to their parent nodes. For clarity, only the step numbers of nodes on the retrospective trajectories are shown. Nodes with the same border color belong to the same trajectory (b) Applying the return-and-explore strategy to the undesirable trajectory

where $\lambda \in (0, 1)$, $DB(\cdot)$ denotes the dual bound of a state, s_e represents the end state of the trajectory, and DB_b and DB_w denote the best (the largest) and the worst (the smallest) dual bounds encountered during solving, respectively. With the normalization in (4), r_t^b is constrained to $[0, 1)$.

The final reward at step t is calculated by

$$r_t = r_t^d + r_t^b. \tag{5}$$

4.3 Return and explore

Given the vast state-action space and the complexity of sequential decision-making in B&B, relying solely on random actions for exploration may be insufficient to yield promising results [35], particularly when learning from scratch. Therefore, an efficient RL branching policy requires a well-designed exploration strategy to identify high-quality training samples. Revisiting Figure 3(a), the trajectory from node 22 to node 27 fails to improve the dual bound in any way. Such trajectories are deemed undesirable, potentially generating a substantial number of samples that are not beneficial for learning an effective policy. To facilitate dual bound improvement and efficiently complete the optimization, the exploration strategy in PEBSI focuses on refining the quality of retrospective trajectories, allowing the agent to acquire more informative training samples.

Consider an undesirable trajectory, the procedure of the strategy is presented in Algorithm 1. It first returns to the starting node of the trajectory, randomly selects different actions at that step, and then finishes the corresponding solving processes again. This generates new subtrees originating from the starting node, enabling the reconstruction of new trajectories. If these new trajectories can improve the dual bound or even terminate the optimization, they are more likely to provide valuable insights for the agent. The trajectory that achieves the greatest dual bound improvement is

selected to replace the original one. Figure 3(b) illustrates the application of this strategy to the undesirable trajectory in Figure 3(a).

Algorithm 1 Exploration strategy of PEBSI

Require: original undesirable trajectory $\tau = (node_1, node_2, \dots)$, action set at the starting node A_s, original action a_0, explored action set $A_e = \{a_0\}$, reconstructed trajectory set $T = \emptyset$, number of action trials N

Replay the original solving process up to $node_1$
for $n = 1, 2, \dots, N$ **do**
    Randomly choose an action $a_n \in A_s \setminus A_e$
    Complete the solving process
    Reconstruct trajectories T_n within the subtree of $node_1$
    $A_e = A_e \cup \{a_n\}$
    $T = T \cup T_n$
end for
Choose $\hat{\tau} \in T$ with the greatest dual bound improvement
Replace τ with $\hat{\tau}$

4.4 Training procedure

Among various RL algorithms, Proximal Policy Optimization (PPO) [36] is widely recognized for its robust performance across diverse tasks. This study employs a specific variant, PPO-Clip, which constrains policy updates to prevent the new policy π_θ from deviating excessively from the old policy $\pi_{\theta_{old}}$. The optimization follows a clipped surrogate objective, given by $L^C(\theta) = \hat{\mathbb{E}}_t[L_t^C(\theta)]$, where

$$L_t^C(\theta) = \min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}\hat{A}_t, \text{clip}\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}, 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right), \tag{6}$$

with $\hat{A}_t$ representing the advantage estimate and ϵ being a hyperparameter.

Since the model in this study shares parameters between the policy and the value function, the objective must include an additional error term $L^V(\theta) = \hat{\mathbb{E}}_t[L_t^V(\theta)]$, where

$$L_t^V(\theta) = (V_\theta(s_t) - \hat{R}_t)^2, \tag{7}$$

with V_θ denoting the value function and $\hat{R}_t$ representing the discounted return. Furthermore, an entropy bonus term $S[\pi_\theta]$ is incorporated into the objective to enhance learning stability. Combined with the exploration strategy in Algorithm 1, the complete training procedure of PEBSI is outlined in Algorithm 2. The sampling process can be executed in parallel to reduce computation time.

Algorithm 2 Training procedure of PEBSI

Require: initial policy parameters θ_0, number of iterations I, number of episodes per iteration K, ϵ, value function error coefficient c_1, entropy bonus coefficient c_2

for $i = 0, 1, 2, \ldots, I-1$ **do**
  for $k = 0, 1, 2, \ldots, K-1$ **do**
    Randomly sample a training instance
    Collect the original episode τ_k by running policy π_{θ_i}
    Construct τ_k and obtain retrospective trajectories
    Compute $\hat{A}_t = \hat{R}_t - V_{\theta_i}(s_t)$ for all the collected samples
  end for
  Update policy parameters θ_{i+1} by maximizing

$$L(\theta) = \hat{\mathbb{E}}_t[L_t^C(\theta) - c_1 L_t^V(\theta) + c_2 S[\pi_\theta](s_t)] \tag{8}$$

end for

5 Experiments

5.1 Experimental setup

5.1.1 Benchmarks

Following common benchmark selection practices in the research community [19, 20, 22–24, 26], PEBSI is evaluated on four classic combinatorial optimization problems: `set covering (Setcover)`, `capacitated facility location (Facility)`, `maximum independent set (Indset)`, and `multiple knapsack (Mknapsack)`. These benchmarks vary in constraint structures, the presence of continuous variables and non-binary integer variables, and optimization directions [37]. They are widely used to model real-world problems [8], making them representative for this study.

For training, a total of 10,000 instances are generated. The policy is then evaluated on two datasets: a test set of 100 instances matching the training problem size (same number of variables and constraints) and a transfer set of 100 instances with a larger problem size (more variables and constraints). Detailed benchmark parameter settings are provided in Appendix A.

5.1.2 Baselines

PEBSI is evaluated against three RL-based baselines that do not rely on expert demonstrations: FMSTS [25], tMDP [26], and Retro [19]. For tMDP, its tree MDP property is activated by providing the optimal objective limit during training. Additionally, for reference, the performance of the IL-based policy trained with SB demonstrations [13] is also reported. All experimental configurations are kept consistent with those specified in the respective papers.

5.1.3 Training and evaluation

PEBSI does not impose specific requirements on the node selection rule during training, and the default solver setting for node selection is maintained in this study. As noted in [19], the choice of construction heuristic has little impact on the performance of the RL agent. In this study, the leaf nodes of the retrospective trajectories are selected randomly. The exploration strategy outlined in Algorithm 1 is applied during the initial 20,000 training epochs, with the number of different actions trials set to 10. The hyperparameter settings are summarized in Table 2. The Adam optimizer is used for optimization.

Table 2 Training parameters

Parameter	Value	Parameter	Value
Number of iterations	15,000	Entropy bonus coefficient	0.01
Number of episodes per iteration	10	Batch size	64
Number of epochs per iteration	5	Actor learning rate	1e-3
λ for calculating r_t^b	0.2	Critic learning rate	3e-4
Value function error coefficient	0.5	Discount factor	0.99

The primary evaluation metric is the final B&B search tree size, which is hardware-independent and effectively reflects the efficiency of the policy. This metric is widely accepted in the research community. Each method is assessed on both the test and transfer sets for comparison. For every instance, three random seeds are used, with a time limit of one hour. The results are presented as mean values, along with the average per-instance standard deviation.

The back-end solver is SCIP [38], one of the fastest open-source MILP solvers. The models are developed using PyTorch [39], PyTorch Geometric [40], PySCIPOpt [41], and Ecole [42] libraries. All experiments are performed on a machine equipped with an Intel(R) Xeon(R) Gold 6348 CPU running at 2.60 GHz and an Nvidia Tesla V100S-PCIE-32GB GPU.

5.2 Results and discussion

5.2.1 Comparative results

Table 3 presents the solving performance of each branching policy across the benchmarks. PEBSI consistently achieves smaller search trees than other RL baselines in nearly all cases and performs comparably to the best method when not leading. While RL-based approaches still lag behind the IL-based method that leverages expert demonstrations, notable exceptions arise. For `Facility`, PEBSI matches and even slightly outperforms the IL approach on the test set. Additionally, for `Mknapsack`, RL-based branching policies surpass IL in most cases, potentially due to the distinct characteristics of different MILPs. Specifically, as previously discussed, the poor LP

relaxation provided by the problem formulation often results in little or no dual bound improvement after branching. This diminishes the informativeness of SB scores, which IL heavily relies on, thereby limiting its performance. These observations highlight the potential of RL as a viable alternative to IL for discovering superior branching strategies across diverse MILPs. It is also noted that Retro exhibits convergence difficulties on `Indset`. A possible explanation lies in its algorithmic structure (off-policy training combined with function approximation and bootstrapping), which can lead to instability and divergence for certain tasks [32].

Table 3 Mean size of the final B&B search tree (lower is better). The best-performing RL method is highlighted in bold

Methods	`Setcover`	`Facility`	`Indset`	`Mknapsack`
IL	$39.57 \pm 8.08\%$	$130.62 \pm 30.66\%$	$34.13 \pm 28.64\%$	$246.49 \pm 70.92\%$
FMSTS	$128.33 \pm 18.55\%$	$169.61 \pm 40.88\%$	$168.92 \pm 60.71\%$	$215.05 \pm 71.38\%$
tMDP	$129.36 \pm 18.40\%$	$164.13 \pm 36.46\%$	$253.70 \pm 53.21\%$	$210.24 \pm 71.57\%$
Retro	$57.38 \pm 12.76\%$	$145.87 \pm 34.49\%$	$4702.72 \pm 56.77\%$	$283.63 \pm 70.96\%$
PEBSI	$\mathbf{51.94 \pm 10.79}\%$	$\mathbf{127.68 \pm 26.48}\%$	$\mathbf{53.32 \pm 32.76}\%$	$\mathbf{201.99 \pm 71.77}\%$

Test

Methods	`Setcover`	`Facility`	`Indset`	`Mknapsack`
IL	$118.87 \pm 9.34\%$	$399.42 \pm 36.20\%$	$254.56 \pm 48.48\%$	$777.41 \pm 67.70\%$
FMSTS	$577.29 \pm 18.54\%$	$583.17 \pm 50.90\%$	$660.69 \pm 65.03\%$	$\mathbf{704.47 \pm 58.21}\%$
tMDP	$599.16 \pm 19.47\%$	$591.34 \pm 51.82\%$	$749.57 \pm 67.64\%$	$828.08 \pm 64.04\%$
Retro	$238.71 \pm 28.71\%$	$624.21 \pm 32.52\%$	$8914.57 \pm 65.12\%$	$775.03 \pm 64.69\%$
PEBSI	$\mathbf{218.20 \pm 13.59}\%$	$\mathbf{471.66 \pm 29.93}\%$	$\mathbf{434.49 \pm 22.91}\%$	$706.94 \pm 64.06\%$

Transfer

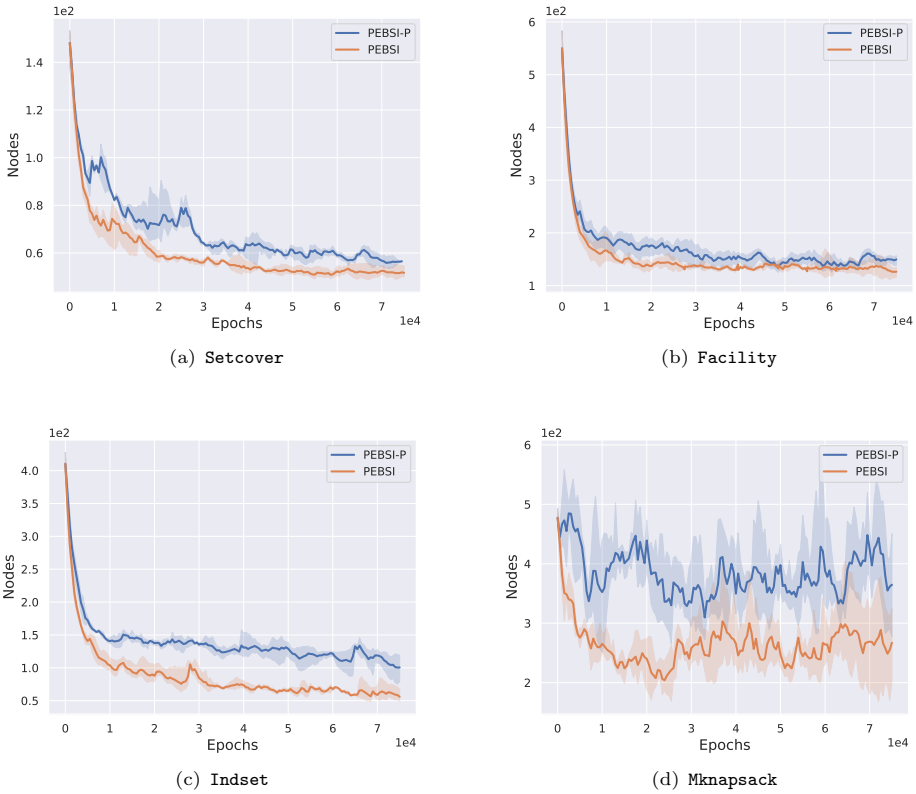
For RL approaches, Table 4 presents the number of training episodes required for convergence on each benchmark, reported as the mean value with standard deviation. The results indicate that PEBSI achieves superior training efficiency compared to the baselines.

Table 4 Number of training episodes (in thousands) required for RL methods to converge. Retro fails to converge on `Indset`. The best-performing method is highlighted in bold

Methods	`Setcover`	`Facility`	`Indset`	`Mknapsack`
FMSTS	$82.00 \pm 26.55\%$	$43.33 \pm 6.06\%$	$112.50 \pm 5.78\%$	$88.33 \pm 10.67\%$
tMDP	$75.00 \pm 37.33\%$	$104.67 \pm 19.80\%$	$113.67 \pm 3.24\%$	$99.67 \pm 22.25\%$
Retro	$70.59 \pm 13.03\%$	$27.29 \pm 56.42\%$	-	$46.17 \pm 15.02\%$
PEBSI	$\mathbf{62.33 \pm 24.90}\%$	$\mathbf{21.33 \pm 34.73}\%$	$\mathbf{63.00 \pm 55.91}\%$	$\mathbf{45.00 \pm 6.67}\%$

5.2.2 Ablation study

To assess the effectiveness of the proposed reward function and exploration strategy, an ablation study was conducted by comparing PEBSI with a plain policy trained with (3) and a stochastic action-sampling exploration strategy (denoted as '-P'). Figure 4 illustrates the evolution of the mean search tree size on the test set during training. The proposed enhancements in PEBSI facilitate deeper exploration, leading to clear improvements in the curves across the benchmarks compared to the plain policy. Figure 5 presents the evolution of undesirable trajectory numbers on the test set during training. The results demonstrate that PEBSI effectively reduces trajectories that fail to improve the dual bound, especially for `Mknapsack`, which inherently generates a substantial number of such trajectories, as discussed earlier.



(a) `Setcover` (b) `Facility`

(c) `Indset` (d) `Mknapsack`

Fig. 4 Evolution of the mean search tree size on the test set during training

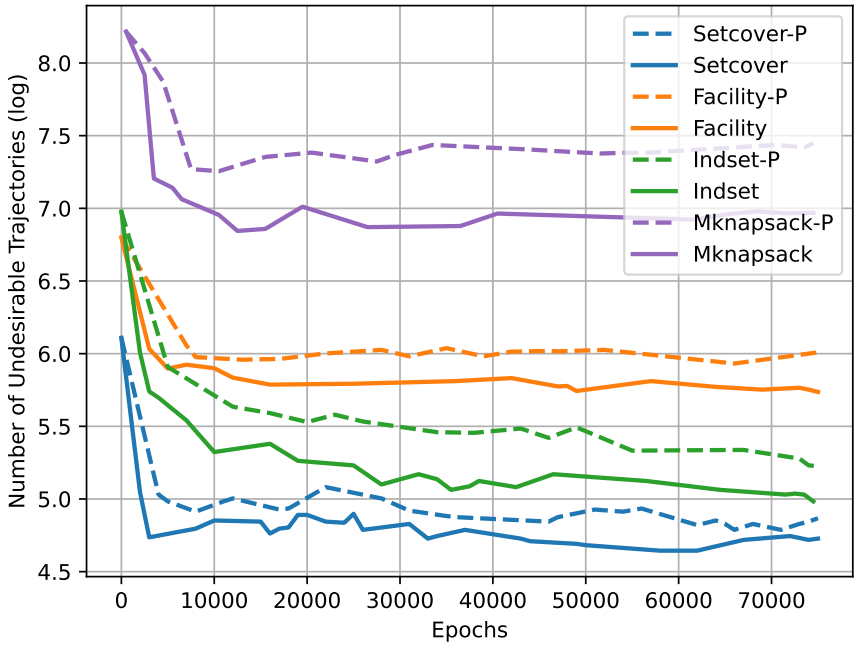


Fig. 5 Evolution of undesirable trajectory numbers on the test set during training, with the y-axis on a log-scale

5.2.3 Informative features

To gain further insight, a qualitative analysis of variable feature importance in branching decisions was conducted, using the gradients of the output score with respect to the input features. The variable features are grouped based on the type of information they convey, as summarized in Table 5. Details of the features are provided in Appendix B. To analyze the dynamics of feature importance during solving, each benchmark was examined with respect to changes across branching steps. Consistent patterns were observed among instances within the same problem set. Figure 6 illustrates one representative instance per benchmark. The results show that feature importance evolves during solving, and different problem types exhibit distinct characteristics.

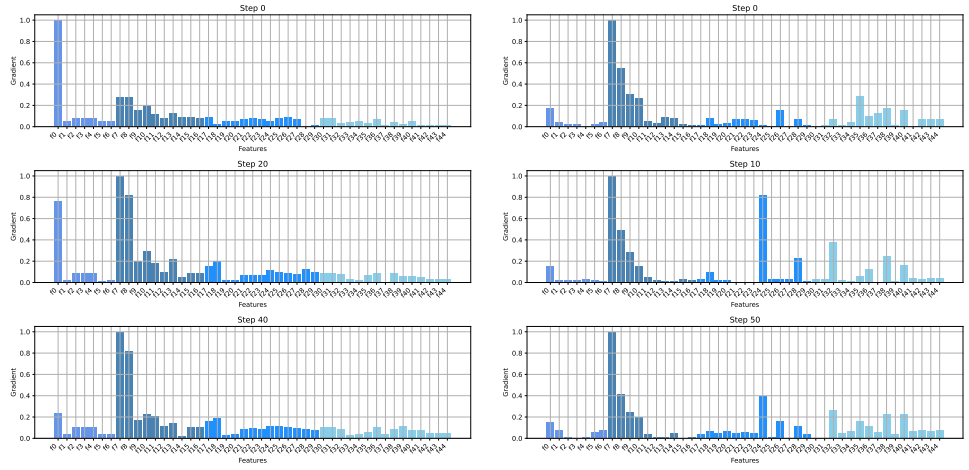
In `Setcover`, the objective coefficient (f_1) is initially influential. As solving progresses, features associated with the LP solution value and its fractionality (f_7, f_8) become more dominant. In `Facility` and `Indset`, LP solutions are also critical. In addition, features that capture structural information, such as number of leaf nodes (f_{24}) and current node depth (f_{32}), can also be important. In `Mknapsack`, uninformative LP-related features tend to be less important, while information about feasible

solutions (f_{28}, f_{29}) plays a more prominent role. This observation is consistent with the design choice of PEBSI to avoid reliance on SB scores and leverage global bound improvement signals to refine the search process.

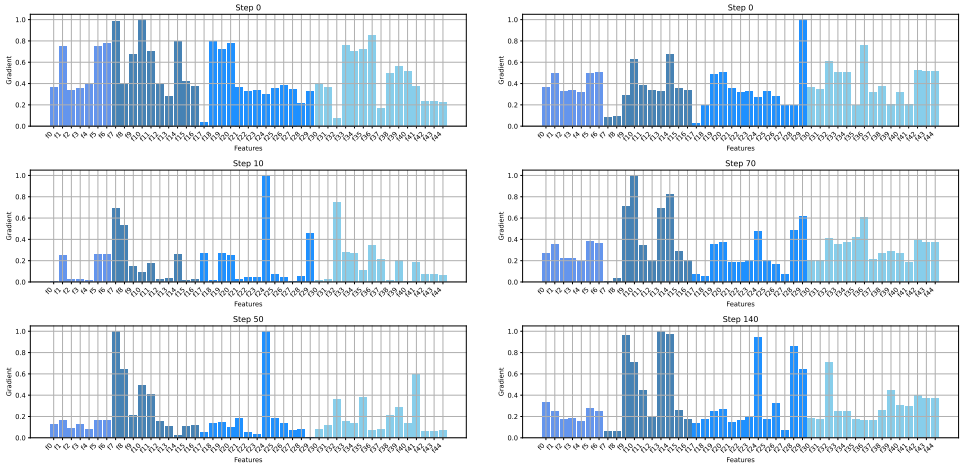
Overall, the analysis highlights the significance of learning a branching policy that adapts to problem-specific characteristics and incorporates diverse information throughout the solving process.

Table 5 Categories of variable features

Category	Features	Description
Problem definition	f_0-f_6	Encode static properties of the original problem, including the objective value coefficient, variable type, bound indicator, etc.
Simplex iteration	f_7-f_{16}	Capture intermediate data from the simplex algorithm, including LP solution, basis status, reduced costs, etc.
Solving process	f_{17}-f_{29}	Represent global solver information, including current feasible and incumbent solutions, number of leaf nodes, etc.
Node information	f_{30}-f_{44}	Reflect properties of the current node, including node depth, optimality status, sibling data, etc.



(a) `Setcover` (b) `Facility`



(c) `Indset` (d) `Mknapsack`

Fig. 6 Variable feature importance (scaled by the maximum value) across branching steps

6 Conclusion

This paper introduces PEBSI, an efficient RL-based policy for branching variable selection, which removes the reliance on expert demonstrations or additional training requirements. The RL agent learns from retrospectively constructed trajectories from the original solving episodes, guided by a novel reward function that captures the varying impact of actions and an exploration strategy designed to identify uninformative trajectories and improve the quality of training samples. Established RL techniques are adapted to the specific setting. Comparative evaluations demonstrate that PEBSI consistently surpasses other RL-based baselines trained without expert

guidance and, in some cases, even outperforms the IL-based approach. An ablation study further validates the effectiveness of the reward function and the exploration strategy, highlighting their contributions to performance improvement. The analysis of variable feature importance provides further insight into the learned policy. This work advances the application of RL in decision-making for B&B procedures and highlights its potential to exceed heuristics grounded in expert knowledge and empirical practices. Future research could explore extending RL to other components of combinatorial optimization, improving sample efficiency and generalization capabilities, and further narrowing the performance gap between RL and its supervised-learning counterparts.

Appendix A Benchmark parameters

The parameter settings for generating the benchmarks are listed in Table 6.

Table 6 Parameter settings for generating the benchmark instances

Benchmark	Parameter	Train/Test	Transfer
`Setcover`	Columns	250	400
	Rows	500	750
`Facility`	Facilities	20	30
	Customers	20	30
`Indset`	Nodes	350	500
	Affinity	4	4
`Mknapsack`	Items	50	100
	Knapsacks	6	12

Appendix B Input features

Based on the bipartite state representation, the constraint and edge features follow the design in [13]. The variable features are extended with 20 additional items from [19], along with six others (f_{28}, f_{29}, f_{35}, f_{36}, f_{37}, and f_{38}) that capture information about current feasible solutions, branching history, and bound updates. Table 7 provides a complete list of the variable features.

References

[1] Liu, Q., Li, X., Gao, L., Fan, J.: A multi-milp model collaborative optimization method for integrated process planning and scheduling problem. IEEE Transactions on Engineering Management **71**, 4574–4586 (2022)

Table 7 List of variable features

Serial number	Feature	Serial number	Feature
f_0	objective	f_{23}	max_pb_frac_change
f_1	is_type_binary	f_{24}	num_leaves_frac
f_2	is_type_integer	f_{25}	num_feasible_leaves_frac
f_3	is_type_implicit_integer	f_{26}	num_infeasible_leaves_frac
f_4	is_type_continuous	f_{27}	num_lp_iterations_frac
f_5	has_lower_bound	f_{28}	num_feas_sol_rel_nodes
f_6	has_upper_bound	f_{29}	num_feas_sol_rel_to_lps
f_7	solution_value	f_{30}	is_curr_node_best
f_8	solution_frac	f_{31}	is_curr_node_parent_best
f_9	is_solution_at_lower_bound	f_{32}	curr_node_depth
f_{10}	is_solution_at_upper_bound	f_{33}	curr_node_db_rel_init_db
f_{11}	scaled_age	f_{34}	curr_node_db_rel_global_db
f_{12}	normed_reduced_cost	f_{35}	branching_change_frac
f_{13}	is_basis_lower	f_{36}	cons_prop_change_frac
f_{14}	is_basis_basic	f_{37}	prop_change_frac
f_{15}	is_basis_upper	f_{38}	pr_branch_frac
f_{16}	is_basis_zero	f_{39}	is_best_sibling_none
f_{17}	incumbent_value	f_{40}	num_siblings_frac
f_{18}	average_incumbent_value	f_{41}	is_best_sibling_best_node
f_{19}	db_frac_change	f_{42}	best_sibling_db_rel_init_db
f_{20}	pb_frac_change	f_{43}	best_sibling_db_rel_global_db
f_{21}	gap_frac	f_{44}	best_sibling_db_rel_curr_node_db
f_{22}	max_db_frac_change		

[2] Adrio, G., García-Villoria, A., Juanpera, M., Pastor, R.: Milp model for the mid-term production planning in a chemical company with non-constant consumption of raw materials. an industrial application. Computers & Chemical Engineering **177**, 108361 (2023)

[3] Bazirha, M.: A novel milp formulation and an efficient heuristic for the vehicle routing problem with lunch break. Annals of Operations Research, 1–26 (2023)

[4] Marlière, G., Richard, S.S., Pellegrini, P., Rodriguez, J.: A conditional time-intervals formulation of the real-time railway traffic management problem. Control Engineering Practice **133**, 105430 (2023)

[5] Land, A.H., Doig, A.G.: An Automatic Method for Solving Discrete Programming

Problems. Berlin Heidelberg, Boston (2010)

[6] Lodi, A.: The heuristic (dark) side of mip solvers. In: Hybrid Metaheuristics (2013). https://api.semanticscholar.org/CorpusID:28868684

[7] Achterberg, T., Wunderling, R.: Mixed integer programming: Analyzing 12 years of progress. (2013). https://api.semanticscholar.org/CorpusID:59830161

[8] Khalil, E.B., Morris, C., Lodi, A.: Mip-gnn: A data-driven framework for guiding combinatorial solvers. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, pp. 10219–10227 (2022)

[9] Applegate, D., Bixby, R., Chvatal, V., Cook, B.: Finding Cuts in the TSP (A preliminary report). Center for Discrete Mathematics & Theoretical Computer Science (1995)

[10] Khalil, E., Le Bodic, P., Song, L., Nemhauser, G., Dilkina, B.: Learning to branch in mixed integer programming. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 30 (2016)

[11] Hansknecht, C., Joormann, I., Stiller, S.: Cuts, primal heuristics, and learning to branch for the time-dependent traveling salesman problem. arXiv preprint arXiv:1805.01415 (2018)

[12] Alvarez, A.M., Louveaux, Q., Wehenkel, L.: A machine learning-based approximation of strong branching. INFORMS Journal on Computing **29**(1), 185–195 (2017)

[13] Gasse, M., Chételat, D., Ferroni, N., Charlin, L., Lodi, A.: Exact combinatorial optimization with graph convolutional neural networks. Advances in neural information processing systems **32** (2019)

[14] Gupta, P., Gasse, M., Khalil, E., Mudigonda, P., Lodi, A., Bengio, Y.: Hybrid models for learning to branch. Advances in neural information processing systems **33**, 18087–18097 (2020)

[15] Nair, V., Bartunov, S., Gimeno, F., Von Glehn, I., Lichocki, P., Lobov, I., O'Donoghue, B., Sonnerat, N., Tjandraatmadja, C., Wang, P., et al.: Solving mixed integer programs using neural networks. arXiv preprint arXiv:2012.13349 (2020)

[16] Achterberg, T., Koch, T., Martin, A.: Branching rules revisited. Operations Research Letters **33**(1), 42–54 (2005)

[17] Berthold, T., Gamrath, G., Salvagnin, D.: Exploiting dual degeneracy in branching. (2019). https://api.semanticscholar.org/CorpusID:149449386

[18] Gamrath, G., Berthold, T., Salvagnin, D.: An exploratory computational analysis

of dual degeneracy in mixed-integer programming. EURO J. Comput. Optim. **8**, 241–261 (2020)

[19] Parsonson, C.W.F., Laterre, A., Barrett, T.D.: Reinforcement learning for branch-and-bound optimisation using retrospective trajectories. In: AAAI Conference on Artificial Intelligence (2022). https://api.semanticscholar.org/CorpusID:249192146

[20] Sun, H., Chen, W., Li, H., Song, L.: Improving learning to branch via reinforcement learning. (2020). https://api.semanticscholar.org/CorpusID:235602520

[21] Qu, Q., Li, X., Zhou, Y., Zeng, J., Yuan, M., Wang, J., Lv, J., Liu, K., Mao, K.: An improved reinforcement learning algorithm for learning to branch. ArXiv **abs/2201.06213** (2022)

[22] Zhang, C., Ouyang, W., Yuan, H., Gong, L., Sun, Y., Guo, Z., Dong, Z., Yan, J.: Towards imitation learning to branch for MIP: A hybrid reinforcement learning based sample augmentation approach. In: The Twelfth International Conference on Learning Representations (2024). https://openreview.net/forum?id=NdcQQ82mfy

[23] Zhang, T., Banitalebi-Dehkordi, A., Zhang, Y.: Deep reinforcement learning for exact combinatorial optimization: Learning to branch. 2022 26th International Conference on Pattern Recognition (ICPR), 3105–3111 (2022)

[24] Huang, Z., Chen, W., Zhang, W., Shi, C., Liu, F., Zhen, H.-L., Yuan, M., Hao, J., Yu, Y., Wang, J.: Branch ranking for efficient mixed-integer programming via offline ranking-based policy learning. In: ECML/PKDD (2022). https://api.semanticscholar.org/CorpusID:252518363

[25] Etheve, M., Alès, Z., Bissuel, C., Juan, O., Kedad-Sidhoum, S.: Reinforcement learning for variable selection in a branch and bound algorithm. ArXiv **abs/2005.10026** (2020)

[26] Scavuzzo, L., Chen, F., Ch'etelat, D., Gasse, M., Lodi, A., Yorke-Smith, N., Aardal, K.I.: Learning to branch with tree mdps. ArXiv **abs/2205.11107** (2022)

[27] Boyd, S.P., Vandenberghe, L.: Convex Optimization. Cambridge university press, Cambridge (2004)

[28] Achterberg, T.: Constraint integer programming (2007)

[29] Bénichou, M., Gauthier, J.-M., Girodet, P., Hentges, G., Ribière, G., Vincent, O.: Experiments in mixed-integer linear programming. Mathematical Programming **1**, 76–94 (1971)

[30] Achterberg, T., Berthold, T.: Hybrid branching. In: Integration of AI and OR

Techniques in Constraint Programming for Combinatorial Optimization Problems: 6th International Conference, CPAIOR 2009 Pittsburgh, PA, USA, May 27-31, 2009 Proceedings 6, pp. 309–311 (2009). Springer

[31] Zarpellon, G., Jo, J., Lodi, A., Bengio, Y.: Parameterizing branch-and-bound search trees to learn branching policies. In: Proceedings of the Aaai Conference on Artificial Intelligence, vol. 35, pp. 3931–3939 (2021)

[32] Sutton, R.S., Barto, A.G.: Reinforcement learning: An introduction. IEEE Trans. Neural Networks **9**, 1054–1054 (1998)

[33] Dakin, R.J.: A tree-search algorithm for mixed integer programming problems. The computer journal **8**(3), 250–255 (1965)

[34] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M.A.: Playing atari with deep reinforcement learning. ArXiv **abs/1312.5602** (2013)

[35] Salimans, T., Chen, R.: Learning montezuma's revenge from a single demonstration. arXiv preprint arXiv:1812.03381 (2018)

[36] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. ArXiv **abs/1707.06347** (2017)

[37] Yilmaz, K., Yorke-Smith, N.: A study of learning search approximation in mixed integer branch and bound: Node selection in scip. Ai **2**(2), 150–178 (2021)

[38] Bolusani, S., Besançon, M., Bestuzheva, K., Chmiela, A., Dionísio, J., Donkiewicz, T., Doornmalen, J., Eifler, L., Ghannam, M., Gleixner, A., Graczyk, C., Halbig, K., Hedtke, I., Hoen, A., Hojny, C., Hulst, R., Kamp, D., Koch, T., Kofler, K., Lentz, J., Manns, J., Mexi, G., Mühmer, E., Pfetsch, M.E., Schlösser, F., Serrano, F., Shinano, Y., Turner, M., Vigerske, S., Weninger, D., Xu, L.: The SCIP Optimization Suite 9.0. Technical report, Optimization Online (February 2024). https://optimization-online.org/2024/02/the-scip-optimization-suite-9-0/

[39] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al.: Pytorch: An imperative style, high-performance deep learning library. Advances in neural information processing systems **32** (2019)

[40] Fey, M., Lenssen, J.E.: Fast graph representation learning with pytorch geometric. arXiv preprint arXiv:1903.02428 (2019)

[41] Maher, S., Miltenberger, M., Pedroso, J.P., Rehfeldt, D., Schwarz, R., Serrano, F.: Pyscipopt: Mathematical programming in python with the scip optimization suite. In: Mathematical Software–ICMS 2016: 5th International Conference, Berlin, Germany, July 11-14, 2016, Proceedings 5, pp. 301–307 (2016). Springer

[42] Prouvost, A., Dumouchelle, J., Scavuzzo, L., Gasse, M., Chételat, D., Lodi, A.: Ecole: A gym-like library for machine learning in combinatorial optimization solvers. In: Learning Meets Combinatorial Algorithms at NeurIPS2020 (2020). https://openreview.net/forum?id=IVc9hqgibyB